Master of Science Thesis in Computer Science
Department of Electrical Engineering, Linköping University, 2024

Semantic Search and Retrieval in Radio Logs

Victor Nossborn

LINKÖPING UNIVERSITY

Master of Science Thesis in Computer Science

Semantic Search and Retrieval in Radio Logs

Victor Nossborn

LiTH-ISY-EX–24/5665–SE

Supervisor: **Oksana Moryakova**
ISY, Linköpings universitet
Mathew Goonewardena
Ericsson

Examiner: **Håkan Johansson**
ISY, Linköpings universitet

Division of Communication Systems
Department of Electrical Engineering
Linköping University
SE-581 83 Linköping, Sweden

Copyright © 2024 Victor Nossborn

Abstract

Troubleshooting radio devices that power modern mobile networks is currently a manual and labour-intensive process, where speed is crucial to minimize network downtime. Therefore, there is a strong interest in building a retrieval system capable of intelligent search and retrieval in radio logs. To facilitate effective retrieval, several retrievers were evaluated using different configurations. First, a RoBERTa language model was further pretrained on a dataset of unannotated radio logs. Then, a proprietary annotated retrieval dataset named the Event Log Retrieval (ELR) dataset was collected and utilized along with the MS MARCO retrieval dataset for training and evaluating the retrieval models. The evaluation compared different retrieval paradigms for log retrieval; evaluated the impact of further pretraining the language model on log data; and investigated which configuration yielded the best performance. The results of the investigation show that the late interaction retrieval paradigm used by the ColBERT model performs best for log retrieval. The results also showed that while further pretraining the language model on logs did improve the representations of log data, it did not improve the performance of the implemented retriever. The investigation into the retrieval datasets showed that fine-tuning on the small ELR dataset is insufficient and that fine-tuning on the larger MS MARCO dataset yielded better performance. The best performance was seen though when first fine-tuning on MS MARCO and then on ELR.

Acknowledgments

I would like to thank all the people who have helped make this thesis possible. In particular, I would like to thank my supervisor Mathew Goonewardena at Ericsson for his continuous feedback and guidance during the writing of this thesis, along with the rest of the Hardware analytics team at Ericsson for their support in collecting data and providing valuable feedback. I would also like to thank my supervisor, Oksana Moryakova, and my examiner Håkan Johansson from ISY for their help in developing the idea and guiding me through the writing of the thesis.

Linköping, May 2024
Victor Nossborn

Contents

Notation **ix**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Related Works . . . 2
1.3 Problem Statement . . . 2
1.4 Delimitation . . . 2
1.5 Thesis Outline . . . 2

2 Background **3**
2.1 Representing Text . . . 3
2.1.1 Tokenisation . . . 3
2.1.2 Embeddings . . . 4
2.2 Language Models . . . 5
2.2.1 Training . . . 6
2.2.2 Metrics . . . 7
2.2.3 BERT . . . 8
2.2.4 RoBERTa . . . 9
2.3 Text Retrieval . . . 9
2.3.1 Traditional Methods . . . 9
2.3.2 Neural Retrieval Paradigms . . . 10
2.3.3 Retrieval Metrics . . . 12
2.3.4 Neural Retrieval Models . . . 13

3 Method **15**
3.1 Overview . . . 15
3.2 Implementation . . . 16
3.2.1 LogRoBERTa . . . 16
3.2.2 SentenceBERT . . . 16
3.2.3 ColBERT . . . 16
3.2.4 Code . . . 17
3.3 Data . . . 18
3.3.1 Radio Logs . . . 18

3.3.2 Wikipedia 19
3.3.3 MS MARCO 19
3.3.4 Event Log Retrieval 20
3.4 Experiments 20
3.4.1 Language Model 21
3.4.2 Retrieval Model 21

4 Results **23**
4.1 Language Model 23
4.1.1 Quantitative 23
4.1.2 Qualitative 24
4.2 Retrieval Model 25
4.2.1 Retrieval Paradigm 25
4.2.2 Different Language Models 26
4.2.3 Fine-Tuning on Retrieval Datasets 27

5 Discussion **29**
5.1 Results 29
5.1.1 Language Model 29
5.1.2 Retrieval Model 30
5.2 Method 32

6 Conclusions **33**
6.1 Future Work 34

A Results **41**
A.1 Language Model 41
A.2 Retrieval Metrics 42
A.2.1 MSMARCO 42
A.2.2 Radio Logs 43

B BERT **45**
B.1 Computational cost 45

Notation

Abbrevations

Abbrevation	Meaning
NLP	Natural Language Processing
LLM	Large Language Model
TR	Text Retrieval
MLM	Masked Language Modelling
CEL	Cross-Entropy Loss
MRR	Mean Reciprocal Rank

1

Introduction

In an increasingly connected world, the need for fast and reliable wireless communications has only grown. This modern connectivity is provided using a complex network of radio devices, all working together to provide the speed and reliability that users have come to expect. To maintain this network and minimise downtime, there is a great need for radio technicians to be able to identify any issues with the radios quickly.

1.1 Motivation

Today, troubleshooting radios is still a very manual process. The radios generate a continuous stream of log data, giving technicians insight into the inner functioning of the radios. When an error occurs, technicians will typically download a large set of these logs leading up to the error, then painstakingly go through them, trying to identify any anomalies that could be causing the error. To help in troubleshooting, technicians already use several different tools that can help find occurrences of known issues in the logs. However, these tools are not very helpful in the early stages before any specific issue has been identified. There is therefore a great interest in building a tool that is more suited for assisting the technician in exploring the logs. A good solution for this problem could be a retrieval system specifically adapted for search and retrieval in radio logs. With such a retrieval system, a user would be able to search for anything they want, expressed as a natural language query, and have the system retrieve the relevant parts of the log. The user could then inspect the result of the query, refine it and search for something else. Through iterative searches, the user could incrementally gain a better understanding of the logs.

1.2 Related Works

Building retrieval systems for text documents is a popular and active area of research in the field of Text Retrieval (TR). Traditional retrieval systems, like Best-Matching 25 (BM25) from (Robertson et al., 1995), use simple statistical methods based on measuring the relative frequencies of keywords, while modern approaches instead rely on using language models like the Bidirectional Encoder Representations from Transformers (BERT) from (Devlin et al., 2019). Some examples of modern retrieval systems based on BERT are SentenceBERT (Reimers and Gurevych, 2019) and Contextualised late-interaction over BERT (ColBERT) from (Khattab and Zaharia, 2020)

1.3 Problem Statement

The primary objective of this thesis is to investigate how to build a retrieval system capable of intelligent search and retrieval in logs. Specifically, the research questions to look into are the following.

1. Can we successfully train a language model on our log data?
2. Which retrieval paradigm is most suitable for log retrieval?
3. How can a retrieval model be trained to enable search through logs?

1.4 Delimitation

This thesis will focus on the problem of text retrieval in radio log files, and will not consider the retrieval of other text types. The thesis will also only consider the case of single-line retrieval, and not on retrieving sets of lines based on interdependencies between them.

1.5 Thesis Outline

The thesis is divided into several different chapters. In Chapter 2, the theoretical background needed for understanding the chosen method and evaluation is given. Chapter 3 introduces the methodology used for answering the research questions, including a description of the data, the implementation and the experiments. The results of the experiments are presented in Chapter 4 and are then discussed in Chapter 5. Chapter 5 also presents a discussion of the chosen methodology. In Chapter 6 the thesis is tied together and the conclusions are presented, along with suggestions for future work.

2

Background

This chapter introduces the theoretical background knowledge the rest of the thesis will use. First, an explanation of the process for representing text is presented, followed by an introduction to large language models, and then the central concepts relating to text retrieval.

2.1 Representing Text

In order for a computer system to be able to operate on text written in natural language, it needs to be able to understand it. This problem is well-studied in the field of Natural Language Processing (NLP), which studies building computer systems that can comprehend, interpret, and generate human language. The challenge of understanding text comes in part from the fact that text data is complex. A text consists of paragraphs, sentences, words, and characters, each of which can vary in length. While this variety is easily understood by humans, it presents a challenge for computers since they operate on discrete numerical data. Thus, the text must first be transformed and represented in a discrete numerical form. Within the field of NLP, specifically within the subfield of language modelling, this is commonly accomplished through a two-step process, illustrated in Figure 2.1. The first step is tokenisation, which splits the text into smaller discrete units. The second step is assigning vector representations to each of these units that represent their semantic meaning, which is referred to as embedding. These steps are explained in more detail in the following subsections.

2.1.1 Tokenisation

The process of splitting text is referred to as *tokenisation*, and the resulting discrete units are called *tokens*. Modern language models like BERT (Devlin et al.,

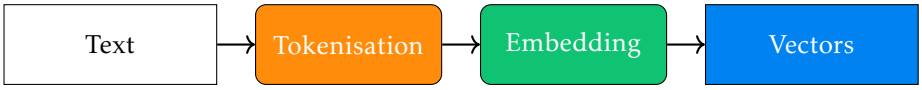


Figure 2.1: *Representing text using tokenisation and embedding.*

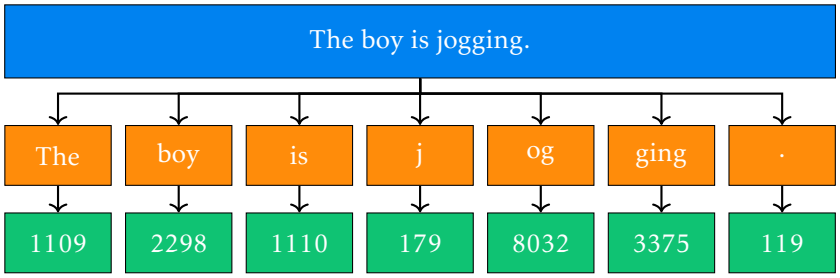


Figure 2.2: *Tokenisation of the sentence "The boy is jogging." using a subword strategy.*

2019) and the Generative Pretrained Transformer 4 (GPT-4) from (OpenAI et al., 2024) use a method called *subword tokenisation*. Specifically, BERT uses the WordPiece tokenisation algorithm while GPT-4 uses Byte-Pair Encoding. An example of subword tokenisation can be seen in Figure 2.2. As seen in the figure, short and common words are typically represented with a single token, while longer and more unusual words are split into separate tokens.

2.1.2 Embeddings

The tokens on their own do not contain any information about their semantic meaning though, and as such are not sufficient to represent the text. In order to represent the semantic meaning of these tokens, a vector representation is needed. The process of assigning vector representation to tokens is called *embedding*, and the resulting vectors *embeddings*. There are several ways to assign these vectors, but in modern NLP the representations are learned from data. The idea of learning semantic representations from tokens was popularised by the Word2Vec (Mikolov et al., 2013) and Global Vectors for Word Representation (GloVe) models (Pennington et al., 2014). Both models trained neural networks to learn representations for their respective sets of tokens. The ambition was to encode the semantic meaning of the tokens into a vector space, which would allow semantically meaningful vector operations to be performed in the vector space. For such a space, one could for example compute that

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}, \tag{2.1}$$

where King, Man, Woman and Queen each correspond to a vector representation. This example is also illustrated in Figure 2.3.

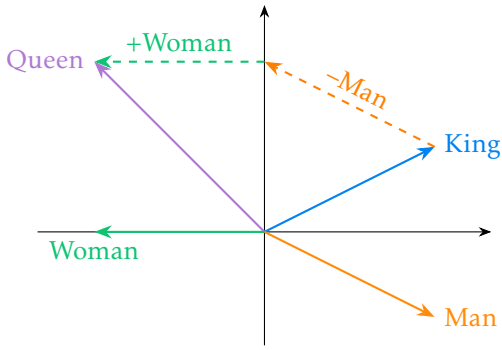


Figure 2.3: *Example showcasing semantically meaningful vector operations that can be performed with embeddings.*

While the Word2Vec and GloVe models saw great success and usage in various NLP tasks, they have one clear problem. This problem is that the semantic meaning of a token can change depending on the other surrounding tokens. Consider the two sentences,

"The *bank* of the river"

and

"Money in the *bank*".

The meaning of *bank* is different in the two sentences. In the former, *bank* is the name for the piece of land next to a river, while in the latter, *bank* refers to a type of financial institution. A solution to this problem called self-attention was proposed by Vaswani et al. (2017). With self-attention, the representation of a token is mixed with the tokens around it, weighted by relevance. For our example, the representation of the token *bank* in each of the sentences is then updated like

$$\text{bank}' = 0.9 \cdot \text{bank} + 0.1 \cdot \text{river} \tag{2.2}$$

and

$$\text{bank}' = 0.9 \cdot \text{bank} + 0.1 \cdot \text{money}, \tag{2.3}$$

thus enabling the same token to take on different meanings depending on the context.

2.2 Language Models

A language model is a model designed to understand human language. These systems typically model language with conditional probabilities over a sequence of tokens. For a sentence like "My favourite food is...", the language model will evaluate all words that might follow, and for each assign a probability based on

how likely it is. For three candidate words, the model can assign the probabilities

$$\begin{aligned} &P(\text{pizza}|\text{My favourite food is}) \approx 0.4, \\ &P(\text{salad}|\text{My favourite food is}) \approx 0.15, \text{ and} \\ &P(\text{rocks}|\text{My favourite food is}) \approx 0.0001. \end{aligned} \tag{2.4}$$

A good language model will understand that pizza being a common favourite food makes it a likely candidate for the next word in this sentence and that while salad is also a food, it is less common as a favourite food and thus receives a lower probability. The final suggestion, "rocks", is not food at all and as such receives a very low probability.

A language model can have a very large number of parameters, such a model is called a Large Language Model (LLM). There are many large language models, with some of the best known being BERT (Devlin et al., 2019), Mistral (Jiang et al., 2023), and GPT-4 (OpenAI et al., 2024). The number of trainable parameters that LLMs have varies, for instance, BERT has hundreds of millions, Mistral tens of billions and GPT-4 is rumoured to exceed a trillion.

For an LLM to be effectively used for a specific problem, it must be trained and evaluated. The following subsections will first outline the process of training an LLM and introduce the metrics used for evaluation. Two prominent LLMs, BERT and RoBERTa, will then be presented.

2.2.1 Training

Training LLMs with hundreds of millions or even billions of parameters from scratch is unfeasible for all but the largest companies and institutions. LLMs like BERT are therefore first released as pretrained *foundation models*, which are intended to be reusable. A foundation model is a model that has not been trained to solve a specific problem but rather has been taught general semantic and world knowledge. These foundation models are intended to be used as starting points for training new, task-specific models.

The training of an LLM for a specific task can therefore be split as shown in Figure 2.4 into two stages: pretraining and fine-tuning. By separating the training process into pretraining and fine-tuning stages, more training resources can be invested in the reusable foundation model. This then enables the creation of downstream models with billions or trillions of parameters at a significantly reduced training cost.

Pretraining

The purpose of pretraining language models is to have a reusable foundation model with general-purpose semantic and world knowledge. Pretraining is different from fine-tuning in the sense that the model is not directly trained on a specific task of interest, but rather on a general learning objective. Typically, the learning objective is chosen to enable unsupervised, or self-supervised, learning which allows the model to train on enormous datasets, without the need for human supervision. BERT is for instance trained on a corpus of books and English

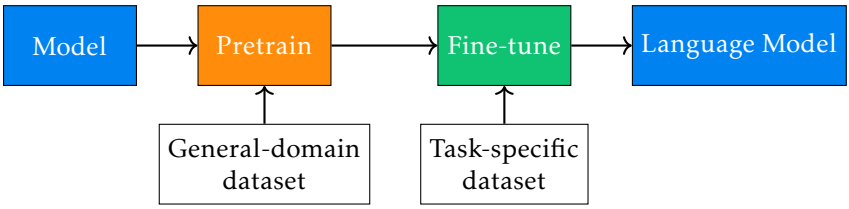


Figure 2.4: *Illustration of the training steps for LLMs on a specific task.*

Wikipedia, totaling around 3.3 billion words (Devlin et al., 2019). The pretrained model can then be modified and used as a starting point for new models that are fine-tuned on a specific task, referred to as a downstream task.

The concept of pretraining became popular in the field of NLP with the introduction of word embedding models like Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014). These models proved useful as building blocks for building new models for different tasks, allowing faster and more reliable training since the model did not have to relearn the semantic meaning of words.

Fine-Tuning

Fine-tuning is the process of training an existing model on a new or modified task. For instance, one can start with a pretrained foundation model and then fine-tune it on a specific problem, like summarising text or detecting sentiment. Unlike pretraining, fine-tuning is often performed supervised with human-labelled datasets. The datasets for fine-tuning are therefore often magnitudes smaller than the ones for pretraining. For example, a common dataset used for passage retrieval, the original MicroSoft MAchine Reading COmprehension (MS MARCO) dataset, consists of around 100 thousand question-answer pairs (Nguyen et al., 2016).

2.2.2 Metrics

There are several ways to evaluate the performance of a language model. A language model is a multinomial classifier that produces a probability distribution over the tokens in the vocabulary. For a multinomial classifier, accuracy and Cross-Entropy Loss (CEL) can be defined.

Cross-Entropy Loss

The Cross-Entropy Loss is defined as

$$l(P, Q) = -\sum_i P(i) \cdot \log Q(i), \tag{2.5}$$

where P is the true distribution and Q is the predicted distribution (Cover and Thomas, 2005). A lower loss corresponds to a predicted distribution that more closely matches the true distribution.

Accuracy

The accuracy metric can be defined as

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}. \tag{2.6}$$

A relaxed extension to the accuracy metric is Top-K accuracy. Top-K accuracy considers not just the class with the highest probability, but rather the top-K most likely classes. If the true class is one of these K classes, then the model prediction is considered correct.

2.2.3 BERT

The Bidirectional Encoder Representations from Transformers (BERT) architecture is an LLM introduced by Devlin et al. (2019) and is commonly used in many NLP problems (Nayak, 2019; Liu and Lapata, 2019). BERT introduced several modifications of the GPT architecture that significantly improved performance on several tasks. The impressive performance of BERT led to the creation of several new models derived from the original, including A Robustly optimized BERT Pretraining approach (RoBERTa) from Liu et al. (2019). The name BERT now commonly refers not only to the original architecture but also to this new family of BERT-like models.

The BERT architecture contributed two major improvements to the GPT architecture. The first was making the model bidirectional, allowing the model to compute attention on both sides of the input token. This allowed the model to form richer, and better contextualised representations for each token. The second major improvement was the introduction of Masked Language Modelling (MLM) as a pretraining objective.

Masked Language Modelling

MLM is a training objective inspired by the Cloze task (Taylor, 1953), commonly used when evaluating the reading comprehension of human students. MLM as a pretraining objective for language models consists of randomly hiding, or "masking", a subset of the input sequence tokens with some probability, which for BERT is 15%. These masked tokens are replaced with the special `<MASK>` token. For example, consider the sentence:

I love eating ice cream on warm, sunny days like this.

In MLM training, each token in this sentence would have a 15% chance of being masked, which might then yield:

I love eating `<MASK>` cream on warm, sunny `<MASK>` like this.

In the masked sentence, the words "ice" and "day" have been replaced by the `<MASK>` token. The language model is then tasked with predicting the masked tokens based only on their context. Given that the original tokens are known, the

Table 2.1: *Performance of BERT and RoBERTa models. F1-score*[1] *is reported for SQuAD and accuracy for MNLI-m and SST-2. Results are taken from the RoBERTa paper (Liu et al., 2019). A higher score implies better performance on the benchmark.*

Model	SQuAD	MNLI-m	SST-2
BERT	90.9/81.8	86.6	93.7
RoBERTa	**94.6/89.4**	**90.2**	**96.4**

MLM objective allows models to train in a self-supervised manner. This enables the model to learn robust and contextual representations of tokens entirely from the data it is trained on.

2.2.4 RoBERTa

RoBERTa introduced by Liu et al. (2019) is an extension of a BERT model with an improved pretraining strategy. In the paper, the authors showed that the original BERT model was significantly undertrained and proposed a set of improvements that significantly increased the performance of the model. The primary modifications were: removing Next-Sentence-Prediction, increasing dataset size from 16GB to 160GB, and switching from static to dynamic masking during pretraining. With these improvements, RoBERTa outperformed BERT and achieved state-of-the-art performance on several benchmarks (Liu et al., 2019). The result for some of these tasks is shown in Table 2.1.

2.3 Text Retrieval

Text Retrieval (TR) is a subfield within information retrieval that studies the problem of retrieving relevant information from text documents (Hambarde and Proença, 2023). A "document" in TR denotes a unit of text of variable length, typically spanning one or a few paragraphs. Text retrieval is commonly used by search engines like Google to find text content that is relevant to users' queries. The system implementation of text retrieval is referred to as a retrieval system, or simply a retriever.

Formally, the text retrieval task can be formulated as an ordinal ranking problem with the goal of ranking documents in order of relevance. In this case, a lower rank implies a higher relevance. The retriever retrieves the top N most relevant documents.

2.3.1 Traditional Methods

Traditional methods for text retrieval typically rely on matching keywords or statistical analysis of the text. One popular traditional method is BM25 introduced

[1] F1-score is a metric balancing precision and recall, see a formal definition in Powers (2008).

by Robertson et al. (1995), which utilises the frequency of tokens in a document. BM25 first tokenises the text, similar to what is described in Section 2.1.1. It does not however embed the text, rather it represents each document as an unordered collection of tokens, with each token having a certain score. The score represents the importance of the token to the document and is based on the frequency of the token in the document, and inversely on the frequency of it in other documents. The relevancy between a query and a document is then determined by comparing the overlap of the collections, weighted by the score of the overlapping tokens.

One major benefit of traditional methods like BM25 lies in their simplicity. They are very computationally efficient, and as such still see much use in simple retrieval systems. Unlike the neural networks described in Section 2.1.2 however, BM25 lacks any semantic understanding of the tokens which means that it is unable to capture complex semantic information like synonyms or pronouns. This means they perform poorly for more complex retrieval problems.

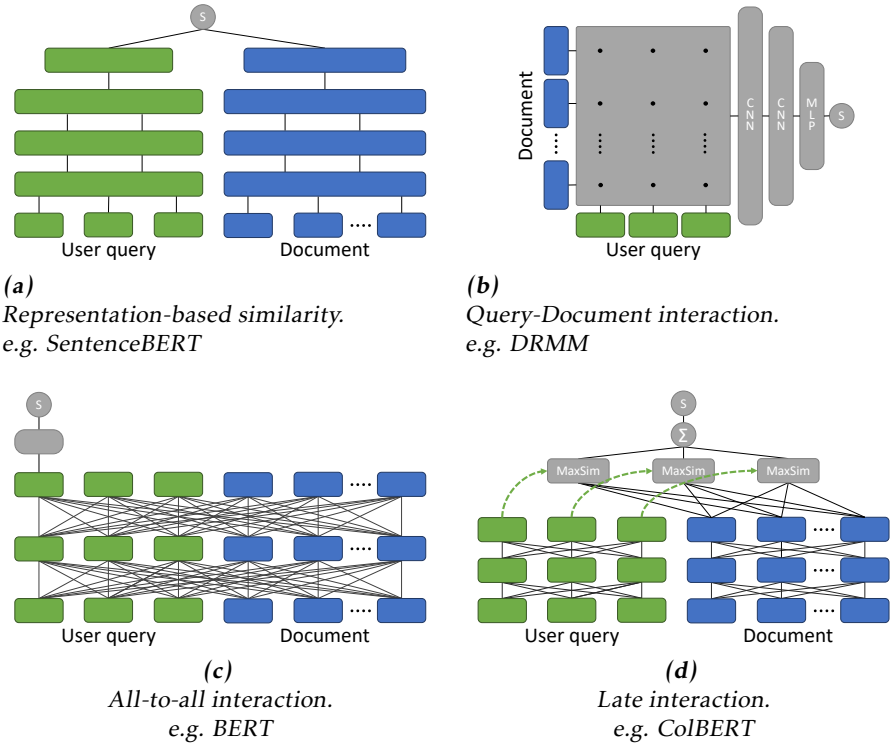
2.3.2 Neural Retrieval Paradigms

Modern alternatives to the traditional methods in information retrieval include various neural network approaches, with recent models having a particular focus on using pretrained language models such as BERT. In order to rank the relevancy between a query and a document, the retriever needs to be able to compare them. Modern retrieval systems differ in how they represent and compare documents and queries, but they can be categorised into four different paradigms (Khattab and Zaharia, 2020) shown in Figure 2.5.

The following subsections detail the strengths and weaknesses of each retrieval paradigm, including their computational scalability in relation to the number of queries and documents, as summarized in Figure 2.2. In these estimations, Q represents a set of queries, D represents a set of documents, and $|D|$ and $|Q|$ represent the total number of documents and queries, respectively. The estimates separate inexpensive vector operations and costly model passthroughs, as the latter require significantly more computational resources. For example, a vector operation may entail a few thousand floating point operations while BERT performs approximately 37 billion operations for a single input, see Section B.1.

Table 2.2: *Summary of the computational scaling of the different retrieval paradigms with regards to the number of documents $|D|$ and queries $|Q|$.*

Paradigm	Model passes	Vector products
Representation-based similarity	$\|Q\| + \|D\|$	$\|Q\| \cdot \|D\|$
Query-Document interaction	$\|Q\| \cdot \|D\|$	–
All-to-all interaction	$\|Q\| \cdot \|D\|$	–
Late interaction	$\|Q\| + \|D\|$	$\sum_{d \in D, q \in Q} \|q\| \cdot \|d\|)$



(a)
Representation-based similarity.
e.g. SentenceBERT

(b)
Query-Document interaction.
e.g. DRMM

(c)
All-to-all interaction.
e.g. BERT

(d)
Late interaction.
e.g. ColBERT

Figure 2.5: *Different query–document matching paradigms in Neural Information Retrieval.*

Representation-based Similarity

The *Representation-based similarity* paradigm, used by models like SentenceBERT (Reimers and Gurevych, 2019) is presented in Figure 2.5a. It involves generating embeddings for the documents and queries independently and then comparing them by computing a single vector similarity. This approach allows the system to precompute all document embeddings in advance, resulting in faster inference. However, summarising entire documents into single embeddings may lead to information loss and doesn't allow any deeper interaction between the document and the query during inference. The total computational cost of this paradigm is then approximately $|Q| + |D|$ expensive model passthroughs and $|Q| \cdot |D|$ cheap vector products.

Query-document Interaction

In contrast, the *Query-document interaction* paradigm shown in Figure 2.5b that is used by models like the Deep Relevance Matching Model (DRMM) by Guo et al. (2016) instead relies on the interaction between the query and document at

the token level. It computes a full interaction matrix between each token in the document and the query, which allows the model to learn more complex relations between queries and documents than the representation-based alternative. The downside, however, is that the model is required to compute the embedding for each query-document pair at inference, which becomes prohibitively expensive as the number of documents grows. The total computational cost of this paradigm is then approximately $|Q| \cdot |D|$ expensive model passthroughs.

All-to-all Interaction

Another full interaction paradigm is the *All-to-all interaction* that is used by models like BERT. This paradigm is shown in Figure 2.5c and works by simply concatenating the query and document and feeding them into the model. Like the query-document interaction, this approach also allows the model to learn complex relations between query and document tokens, but at the cost of computing expensive embeddings for each query-document pair at inference time. The total computational cost of this paradigm is then also approximately $|Q| \cdot |D|$ expensive model passthroughs.

Late Interaction

The final paradigm shown in Figure 2.5d is a *late interaction* approach, introduced by Khattab and Zaharia (2020) with their ColBERT model. This approach attempts to strike a balance between the representation-based and interaction-based approaches by initially embedding the query and document separately as in the the representation-based approach. However, instead of summarising the document and query into single vectors, each token embedding is preserved. This allows an interaction between the query and document tokens to still happen in the latter stages. This combination means that document embeddings can be precomputed as in the representation-based approach, thereby retaining the computational benefit, while maintaining the deep interaction between the query and document found in the interaction-based approaches. The total computational cost of this paradigm is then approximately $|Q| + |D|$ expensive model passthroughs and $\sum_{q \in Q, d \in D} |q| \cdot |d|$ cheap vector products, where $|q|$ and $|d|$ are number of tokens in query q and document d respectively.

2.3.3 Retrieval Metrics

Retrievers are typically evaluated using a set of standard metrics. Two common metrics are the Mean Reciprocal Rank (MMR) and Recall. When evaluating retrieval systems, these metrics are typically restricted to only considering the top-K results. The metrics with these restrictions are then called MRR@K and Recall@K.

Mean Reciprocal Rank

Mean Reciprocal Rank is a measurement for the relevancy of the retrieved results, with an emphasis on the order in which the results were retrieved (Craswell, 2009). The rank of an item is defined as its position in the order, meaning that the item at position r also has the rank r. MRR is defined as

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{r_i}, \tag{2.7}$$

where Q is a set of queries and r_i is the rank of the first relevant result to query i. If the rank is larger than K for MRR@K, the reciprocal $\frac{1}{r}$ is set to 0.

Recall

Recall is a measurement for the relevancy of the retrieved results (Powers, 2008), but unlike MRR, it does not consider order. It instead considers the total number of relevant results that were retrieved. Recall is defined by

$$\text{Recall} = \frac{\text{Number of relevant retrieved items}}{\text{Total number of relevant items}}. \tag{2.8}$$

2.3.4 Neural Retrieval Models

This section introduces two common neural retrieval models, SentenceBERT and ColBERT.

SentenceBERT

SentenceBERT, introduced by Reimers and Gurevych (2019), is a modification of BERT designed to produce semantically meaningful embeddings for entire sentences. SentenceBERT uses a siamese network structure illustrated in Figure 2.6, and is designed to create fixed-size sentence embeddings. The model can be used as a retriever, under the representation-based similarity paradigm, by computing the similarity between query and document embeddings and then retrieving the documents with the highest similarity.

ColBERT

ColBERT is a retrieval model introduced by Khattab and Zaharia (2020) and is a late interaction model architecture. ColBERT, illustrated in Figure 2.7, is designed to leverage the strength of BERT for efficient and effective document retrieval. Unlike traditional retrieval models, ColBERT uses a late interaction mechanism that allows it to balance deep contextualization and scalability. It encodes queries and documents independently into token-level embeddings using BERT and then employs a late interaction phase to compute similarity scores between query and document embeddings. This enables ColBERT to capture fine-grained token interactions while maintaining efficiency during retrieval.

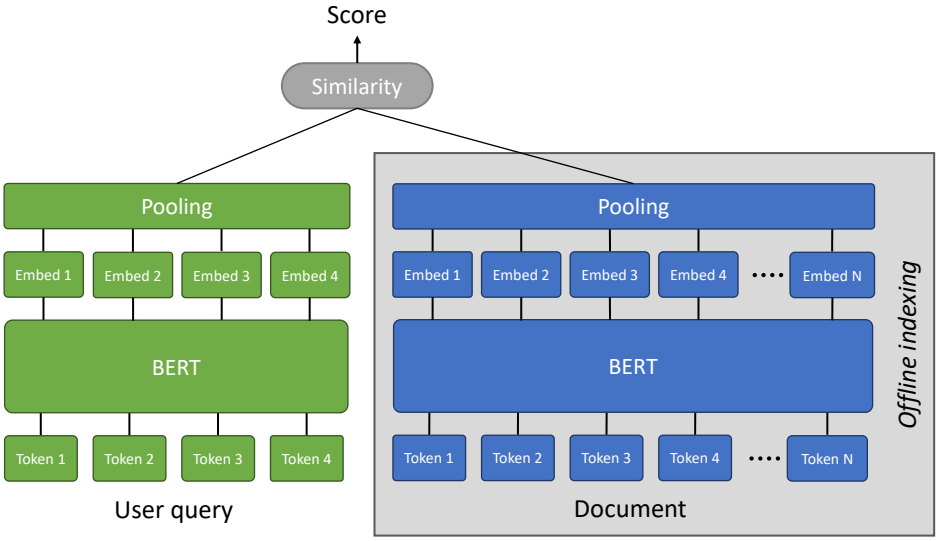


Figure 2.6: *Architecture of the SentenceBERT model. The query and document are encoded separately and compared using a vector similarity.*

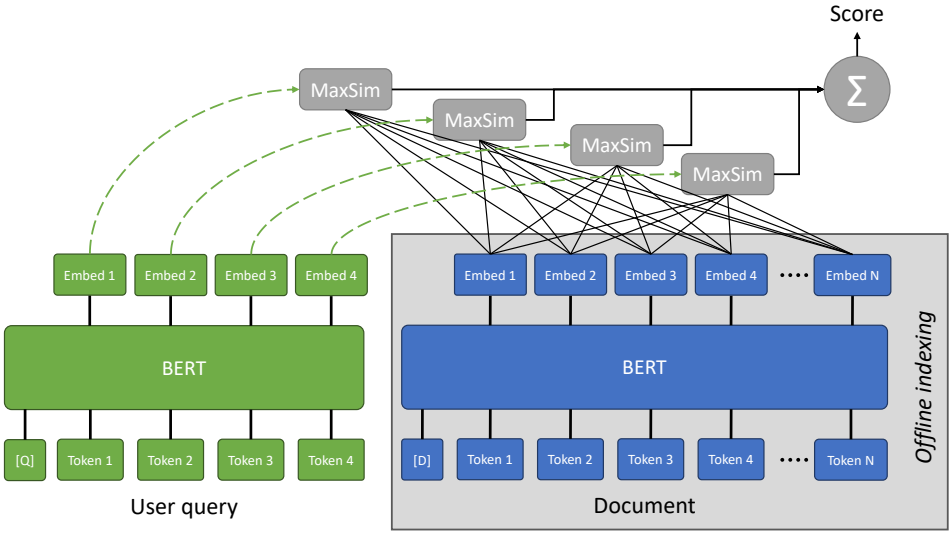


Figure 2.7: *Architecture of the ColBERT model. The query and log lines are encoded separately and compared using maximum similarity.*

3

Method

This chapter outlines the methodology used to answer the research questions from Section 1.3. It begins with an overview of the methodology, followed by detailed descriptions of the implementation of the language and retrieval models. Next, the datasets used for training and evaluating the models are presented. The chapter concludes with a description of the experiments conducted to answer the research questions.

3.1 Overview

The purpose of this thesis is to investigate how to build a retrieval system for radio logs. The methodology is divided into two primary parts: the training and evaluation of a language model on radio logs, and the implementation of a retrieval model for log retrieval.

The first part involves implementing and training a language model. A suitable language model is a prerequisite for building a good retrieval system, so an evaluation is performed to determine if further pretraining on log data can improve token representations. For this purpose, a dataset comprising radio logs is collected and used together with a Wikipedia dataset for training and evaluation.

The second part focuses on which is the most suitable retrieval implementation for log retrieval. There are several options when implementing a retrieval system, including different paradigms, language models, and datasets for fine-tuning. Retrieval models with these different options are implemented and evaluated to identify which configuration yields the best performance. For the training and evaluation, a dataset of human-annotated radio logs is collected and used along with the MS MARCO dataset.

3.2 Implementation

Before the evaluation can take place, several different language and retrieval models are needed. This section details the implementation steps of these models.

3.2.1 LogRoBERTa

The language model that is trained for this thesis is called LogRoBERTa, and it is based on the large version of a pretrained RoBERTa model. The original RoBERTa model, described in Section 2.2.4, is available via Hugging Face (Facebook AI, 2019). The model is further pretrained on the Radio logs dataset to learn to better represent logs.

The training setup is based on the one used in the original paper (Liu et al., 2019), but with some modifications to the learning rate and batch size, as suggested by Gururangan et al. (2020) for continued pretraining. The hyperparameters along with the modifications are detailed in Table 3.1. The LogRoBERTa model is pretrained on the dataset for 2 full epochs.

Table 3.1: *Hyperparameters for training LogRoBERTa, relative to RoBERTa.*

Hyperparameter	LogRoBERTa	RoBERTa
Learning Rate Schedule	Linear	Linear
Peak Learning Rate	1e-5	4e-4
Warmup ratio	6%	6%
Weight decay	0.01	0.01
Batch size	256	8K
Context length	512	512
ϵ	1e-12	1e-6

The preprocessing of the input to LogRoBERTa is kept similar to the original RoBERTa model. The only preprocessing is making all characters lowercase as well as removing redundant white spaces. The preprocessing serves to simplify learning and optimize context usage.

3.2.2 SentenceBERT

The SentenceBERT model used is the `msmarco-MiniLM-L6-cos-v5` model, available via Huggingface (Reimers, 2021a). As described in Section 2.3.2, SentenceBERT follows the representation-based similarity paradigm. As with LogRoBERTa, the only preprocessing is making all characters lowercase as well as removing redundant white spaces.

3.2.3 ColBERT

The ColBERT architecture is implemented in five different variants as shown in Figure 3.1. The implementation of each variant starts with a pretrained language

model and can then follow four possible implementation steps, described below.

- Step 1: Pretrain the language model on the event logs.
- Step 2: Integrate the language model into a ColBERT architecture.
- Step 3: Fine-tune ColBERT on the MS MARCO dataset.
- Step 4: Fine-tune ColBERT on the Event Log Retrieval (ELR) dataset.

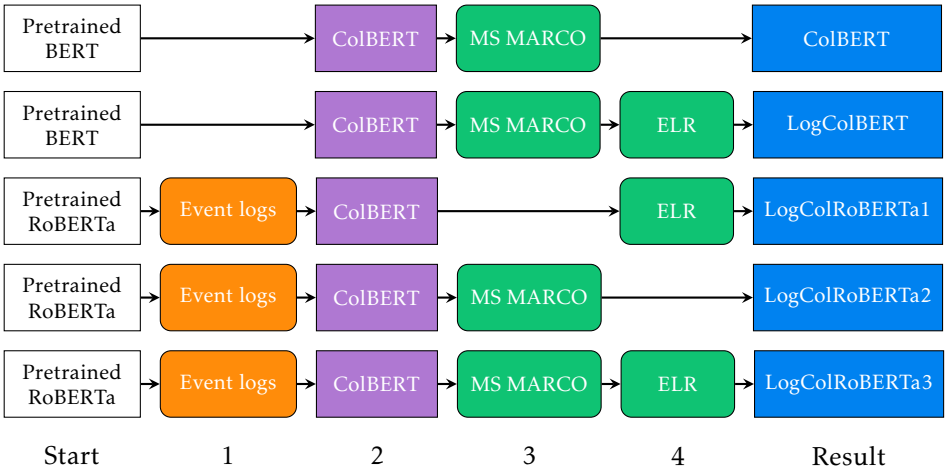


Figure 3.1: *The evaluated variants of the ColBERT implementation, are separated horizontally by the implementation steps.*

The first two models, ColBERT and LogColBERT are implemented using a trained ColBERT model provided by the original authors. This models implementation corresponds to steps 2 and 3. The LogColBERT model is then further fine-tuned on the ELR dataset.

The remaining three models are the LogColRoBERTa models. They in contrast start with a pretrained RoBERTa base model that is further pretrained on the radio logs. These models are then integrated into the ColBERT architecture. The LogColRoBERTa1 model is only fine-tuned on the ELR dataset, while LogColRoBERTa2 is only fine-tuned on the MS MARCO dataset. The final LogColRoBERTa3 model is fine-tuned first on MS MARCO, and then on ELR.

3.2.4 Code

This section briefly explains the code implementation of the language and retrieval models. All models and retrievers are implemented in Python and use `PyTorch` (Paszke et al., 2019) for the machine learning operations. The language models RoBERTa and LogRoBERTa are implemented using the `transformers` library from Huggingface (Wolf et al., 2020) and trained with the library's Trainer

API. The SentenceBERT model is instead implemented using a library called `sentence-transformers` (Reimers, 2021c) and then integrated as a retrieval model using the `LangChain` library (Chase, 2022). The ColBERT retrieval model is implemented using the code provided in the GitHub repository[1] by the original authors, which uses the language model implementations from the `transformers` library. Both retrieval models use the Facebook AI Similarity Search (`FAISS`) vector store from Douze et al. (2024) for storing the document embeddings.

3.3 Data

For training and evaluating the different models, several different datasets are needed. These datasets are shown in Table 3.2 and are further explained in the following subsections. The datasets are split into training and evaluation to ensure that the evaluation is fair. The training splits are only used for training the models, and the evaluation splits are only used for evaluation.

Table 3.2: *Datasets used for training and evaluating models. Note that the MS MARCO evaluation dataset is a subsampled version of the original.*

Dataset	Split	Description
Radio logs	Train	37.1M tokens.
	Eval	892K tokens.
Wikipedia	Eval	260K tokens.
ELR	Train	249 triplets.
	Eval	27 relevancy pairs, 1585 documents, 9 queries.
MS MARCO	Train	39.8M triplets.
	Eval	1000 relevancy pairs, 150K documents, 937 queries.

3.3.1 Radio Logs

In order to pretrain a language model on radio log data, a dataset of radio logs was collected. The Radio logs dataset consists of a series of radio event logs from several different Ericsson radio devices. A description of how each line in these logs is formatted is shown in Figure 3.2.

```
[DATE TIME] CODE: CODE_NAME: DESCRIPTION OF EVENT
```

Figure 3.2: *Format of a line in the Radio logs dataset.*

[1] `https://github.com/stanford-futuredata/ColBERT`

Each logline in the dataset is tokenised into a list of tokens. If the list is longer than 512 tokens, the maximum context length of the BERT and RoBERTa models, then the list is further split into several equal-length parts. The training split of the Radio logs dataset is derived from 20 separate radio log files and totals 37 115 687 tokens. The held-out evaluation split of the dataset comes from one log file and totals 891 674 tokens.

3.3.2 Wikipedia

The Wikipedia dataset is derived from the publicly available dump from English Wikipedia (Wikimedia Foundation, 2018), and consists of paragraphs from Wikipedia articles. An example of a paragraph from the Wikipedia dataset is shown in Figure 3.3. The Wikipedia evaluation dataset totals 260 148 tokens.

```
in 1996, churchill moved to the new ethnic studies
department of the university of colorado.  in 1997, he was
promoted to full professor.  he was selected as chairman of
the department in june 2002.
```

Figure 3.3: *Example of a paragraph from the Wikipedia dataset.*

This dataset is the same as the one used to train and evaluate the original BERT and RoBERTa models, which makes it useful for measuring how further pretraining on a new domain impacts the models performance on the original domain.

3.3.3 MS MARCO

MS MARCO is an open, general domain retrieval dataset provided by Microsoft (Nguyen et al., 2016). MS MARCO contains a few different datasets, but in this thesis, it is the 2020 passage retrieval dataset that is used. This dataset contains three parts. The first is a large set of documents, referred to as the collection. The collection is formatted as seen in Figure 3.4.

```
0 The presence of communication amid scientific minds was
equally important to the success of the Manhattan Project
as scientific intellect was.  The only cloud hanging over
the impressive achievement of the atomic researchers and
engineers is what their success truly meant; hundreds of
thousands of innocent lives obliterated.
```

Figure 3.4: *Format of a document in the document collection. Each line contains an ID and a document.*

The second is a set of queries that are relevant to the documents in the collection. An example of such a query is shown in Figure 3.5, which is the relevant query to the document in Figure 3.4. Note that while each query has one or more relevant documents, not all documents have a relevant query.

```
10 what was the immediate impact of the success of the
manhattan project?.
```

Figure 3.5: *Format of a query in the queries. Each line consists of an ID and a query.*

The third and final part of the dataset combines these queries and documents into contrastive triplets. A contrastive triplet is an association between a query, a relevant document and an irrelevant document. A relevancy pair is the same as a triplet but without an irrelevant document.

The machine on which the evaluation was performed has a limited amount of memory and computation, which is insufficient for the full MS MARCO evaluation dataset consisting of 59,273 relevancy pairs and 8.8 million documents. Therefore, the full evaluation dataset is uniformly subsampled to 1000 relevancy pairs and 150,000 documents.

3.3.4 Event Log Retrieval

The ELR dataset is a propriety dataset collected for this thesis. The purpose of the dataset is to be able to train and evaluate retrieval models for the log retrieval task. The dataset contains human-readable descriptions of loglines in a set of radio logs, contributed by subject matter experts. The dataset is formatted in the same way as described for MS MARCO, with each log line constituting a document, each annotation a query and the association a relevancy pair. The irrelevant documents needed to form triplets were uniformly sampled from the entire collection.

The annotations were collected through a tool where subject matter experts were presented with several log files. For each file, the human annotator went through and selected sets of lines they deemed as interesting. For each of these lines or sets of lines, they provided a natural language text explanation of what they meant.

3.4 Experiments

Several experiments are performed to investigate how to build a retrieval system for radio logs. These experiments are divided into two parts. The first part addresses the possibility of training a language model on log data, corresponding to the first research question. The second part addresses the second and third research questions, focusing on the implementation of a retrieval system. This implementation is investigated through further experiments evaluating the different retrieval paradigms and the impact the different implementation steps have on the final retrieval performance. All experiments and evaluations are performed using an NVIDIA T4 GPU with 16 GB of VRAM.

3.4.1 Language Model

The first experiment investigates if a language model can improve its representation of radio logs by further pretraining on the Radio logs dataset. The experiment then further investigates how improvements to the representations of log data affect the representation of natural language text.

The experiment is carried out by first pretraining a LogRoBERTa model and then comparing its performance to the original RoBERTa model. The performance of the two models is then evaluated on both the Radio logs and Wikipedia evaluation datasets. The two models are evaluated both quantitatively and qualitatively. The quantitative evaluation is performed by comparing the Top-K accuracy and the cross-entropy loss, both described in Section 2.2.2, for both models. The qualitative evaluation is performed by inspecting the predictions of three sample inputs: one representing a common logline, one a rare log line and one normal natural language sentence.

3.4.2 Retrieval Model

In order to investigate what the best implementation of a retrieval system is, several experiments are performed. The first experiment compares the retrieval paradigms against each other. The second experiment then compares the impact of replacing the language model. The final experiment evaluates the performance gained by fine-tuning the models on the different retrieval datasets. The performance comparison between the retrievers in each experiment is performed using the retrieval metrics described in Section 2.3.3.

Retrieval Paradigm

In Section 2.3.2, four retrieval paradigms are introduced. To select the best paradigm for the log retrieval task, the performance of these paradigms is investigated. As described in Section 2.3.2, the two full-interaction paradigms: All-to-all and Query-document interaction, both have poor scaling properties with regard to the number of documents. This makes them unsuitable for the log retrieval task, and as such are not further considered in the paradigm experiments. Instead, the experiment focuses on comparing the two scalable paradigms: the representation-based similarity used by SentenceBERT and the late interaction used by ColBERT. This comparison is performed by first implementing a ColBERT and a SentenceBERT model, that have only been trained on MS-MARCO. Then, their performance is compared on both MS MARCO and the log retrieval task.

Different Language Models

The retrieval architectures rely on a language model to use for generating document and query embeddings. These models are trained on natural text and not logs. An experiment is therefore performed to evaluate if replacing the language model of the architecture with the LogRoBERTa model that has been specifically

trained on radio logs improves the performance of the retriever. This experiment is carried out by first training two BERT-based retrievers: ColBERT and LogColBERT, followed by training the two equivalent LogRoBERTa-based retrievers: LogColRoBERTa2 and LogColRoBERTa3. Finally, their performance is evaluated and compared on both MS MARCO and the log retrieval task.

Fine-Tuning on Retrieval Datasets

There are large general domain retrieval datasets like MS MARCO publicly available. It is therefore an option to fine-tune the retrieval model on the larger MS MARCO dataset instead of the smaller ELR dataset. This experiment is therefore carried out by first training a retrieval model only on ELR, LogColRoBERTa1, and then training a model only on MS MARCO, LogColRoBERTa2. Then, the two models are compared and evaluated on both MS MARCO and ELR.

Another option is to fine-tune the retriever on both MS MARCO and ELR. Creating a new proprietary dataset requires an expert to spend time on annotating data, which is both laborious and expensive, so it is interesting to investigate if the performance meaningfully improves by collecting it. A final experiment is therefore performed by first training two models only on MS MARCO: ColBERT and LogColRoBERTa, then training two models on both MS MARCO and on ELR: LogColBERT and LogColRoBERTa3. These models are then compared and evaluated on both MS MARCO and ELR.

4

Results

In this chapter, the results of the experiments outlined in Section 3.4 are presented.

4.1 Language Model

The results of the experiments on the language model are presented in two parts. First, the quantitative results are presented by comparing the models on evaluation datasets. Second, the qualitative results are presented by comparing the models for three examples.

4.1.1 Quantitative

The Cross-entropy loss and Top-K accuracy (Section 2.2.2) are presented for the original RoBERTa and LogRoBERTa in Table 4.1 for both the Wikipedia and unannotated radio logs datasets.

Table 4.1: *The CLE (Loss) and Top-K accuracy (Top-K) for both models on the evaluation datasets. Lower loss and higher accuracy correspond to better performance.*

Dataset	Model	Loss	Top-1	Top-3	Top-10	Top-50
Radio logs	RoBERTa	3.61	0.408	0.473	0.542	0.691
	LogRoBERTa	**0.449**	**0.854**	**0.893**	**0.942**	**0.985**
Wikipedia	RoBERTa	**1.66**	**0.649**	**0.775**	**0.837**	**0.896**
	LogRoBERTa	2.67	0.464	0.610	0.722	0.814

By first comparing the loss between the models on the radio log evaluation dataset, it is seen that continuing pretraining on radio logs causes the loss to decrease by 87.6%, going from 3.61 down to 0.449. It also increases the model accuracy for all K:s, with the biggest increases seen for low values of K. At $K = 1$ the accuracy increases by 109.3%, at $K = 10$ by 88.8% and at $K = 50$ by 42.5%.

This improvement in performance on radio logs does however correspond to a decrease in performance on the Wikipedia evaluation dataset. Comparing the loss again, continuing pretraining increases the evaluation loss by 60.8%, going from 1.66 up to 2.67. Similarly, the accuracy decreases for all K with the biggest difference seen for low values of K. At $K = 1$ the accuracy decreases by 28.5%, at $K = 10$ by 13.7% and at $K = 50$ by 9.2%.

4.1.2 Qualitative

The RoBERTa and LogRoBERTa performance is compared using the predictions on a few sample inputs. The input is masked and the model generates predictions using the MLM objective as described in Section 2.2.3. For each prediction, the model also outputs a confidence score expressed as a percentage, which is presented next to the prediction in the tables.

The first example is seen in Table 4.2 and represents a common line found in the radio logs. For this line, LogRoBERTa correctly predicts the masked token with 99.9% confidence. RoBERTa however, fails to make the right prediction and shows low certainty in its top three predictions. The top prediction "Error" has a small confidence of 3.82%. Furthermore, while the predictions of RoBERTa would fit grammatically if the line is read like natural language, it is not valid as a line from a radio log.

Table 4.2: *Top-3 masked token predictions and confidence scores for a common radio logline.*

[210507 161321] 49: Throttling <MASK>, elog ID 92, count 0			
Correct	continues		
RoBERTa	Error [3.82%]	off [2.63%]	error [2.30%]
LogRoBERTa	continues [99.9%]	ended [0.13%]	continued [0.01%]

The second example in Table 4.3 represents an uncommon type of logline not well represented in the radio logs dataset. For this line, the LogRoBERTa model still correctly predicts "delta" with a 79.1% confidence. RoBERTa however again makes the wrong prediction, "CPU" with low confidence of 4.88%. It is worth noting that the second and third predictions for both models correctly identify that the line is giving information about a power reading, with LogRoBERTa correctly inferring that it is specifically about voltage.

The third example is seen in Table 4.4 and represents a normal English sentence, with the word "ice" masked. As is seen, the RoBERTa and LogRoBERTa models both successfully predict the masked-out token with 97.5% and 95.9% confidence respectively. Furthermore, the other two suggestions for both models

Table 4.3: *Top-3 masked token predictions and confidence scores for a rare radio logline.*

[211125 090428] 144: Unknown event. Normal Mode; <MASK> U over 5 min...			
Correct	delta		
RoBERTa	CPU [4.88%]	Power [4.26%]	Current [3.07%]
LogRoBERTa	delta [79.1%]	voltage [5.78%]	power [4.45%]

also fit grammatically and conceptually in the sentence. It can be noted however that the LogRoBERTa model shows more uncertainty relative to RoBERTa, with LogRoBERTa shifting some of the probability over from the most to the second most likely token, "whipped".

Table 4.4: *Top-3 masked token predictions and confidence scores for a natural language sentence.*

It is nice to eat some <MASK> cream in the summer.			
Correct	ice		
RoBERTa	ice [97.6%]	whipped [0.39%]	fresh [0.21%]
LogRoBERTa	ice [95.9%]	whipped [2.99%]	sour [0.24%]

4.2 Retrieval Model

This section presents the results of the experiments on the retrieval models on the retrieval datasets. The full results are available in the Appendix, Section A.2.

4.2.1 Retrieval Paradigm

The first retriever experiment compares the performance of a retriever following the late interaction paradigm, ColBERT, with one following the representation-based similarity paradigm, SentenceBERT. The results for both retrieval models on the ELR evaluation dataset can be seen in Table 4.5. As is seen in the table, the interaction-based ColBERT outperforms the representation similarity-based approach on all metrics, with the MRR for ColBERT being approximately 50% higher and recall more than double, for both $K = 10$ and $K = 100$.

For comparison, the same evaluation is performed on the MS MARCO dataset as well. As is seen in Table 4.6, SentenceBERT and ColBERT perform similarly on recall but SentenceBERT achieves a much higher MRR. The MRR result for SentenceBERT is significantly higher than what is reported by the authors of the model (Reimers and Gurevych, 2019), see discussion in Section 5.1.2.

Table 4.5: *Performance of late interaction model (ColBERT) and representation similarity-based model (SentenceBERT) on the ELR evaluation dataset. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
SentenceBERT	0.148	0.154	0.250	0.365
ColBERT	**0.185**	**0.193**	**0.519**	**0.765**

Table 4.6: *Performance of late interaction model (ColBERT) and representation similarity-based model (SentenceBERT) on the MS MARCO evaluation dataset. Higher is better. *Unusually high, see discussion (Section 5.1.2).*

Model	MRR@10	MRR@100	Recall@10	Recall@100
SentenceBERT	**0.860***	**0.861***	0.967	0.996
ColBERT	0.467	0.467	**0.980**	**0.997**

4.2.2 Different Language Models

The second experiment for the retrieval models investigates the impact of replacing the language model with one that has been pretrained on logs. For this comparison, two ColBERT models based on the original BERT model: ColBERT and LogColBERT, are compared to two ColBERT models using the RoBERTa model that has been further pretrained on radio logs, LogColRoBERTa2 and LogColRoBERTa3. ColBERT and LogColRoBERTa2 have only been fine-tuned on MS MARCO, while LogColBERT and LogColRoBERTa3 have been fine-tuned on both the MS MARCO and the ELR datasets.

On the MS MARCO evaluation dataset, the difference is clear. As is seen in Table 4.7, pretraining on radio logs lowers performance on the natural language retrieval task. The BERT-based models ColBERT and LogColBERT outperform the LogRoBERTa-based LogColRoBERTa2 and LogColRoBERTa3 models on both MRR and recall at both $K = 10$ and $K = 100$.

Table 4.7: *Retrieval metrics for the retrieval models based on BERT and LogRoBERTa on the MS MARCO evaluation dataset. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
ColBERT	**0.467**	**0.467**	**0.980**	**0.997**
LogColRoBERTa2	0.436	0.437	0.937	0.962
LogColBERT	**0.467**	**0.468**	**0.979**	**0.998**
LogColRoBERTa3	0.437	0.438	0.938	0.965

On the radio log retrieval task, a similar result is seen. By comparing the relative performance of ColBERT to LogColRoBERTa2, and LogColBERT to LogColRoBERTa3 on the ELR dataset in Table 4.8, it is seen that replacing BERT

with LogRoBERTa does not improve performance on the log retrieval task. Notably, the models with LogRoBERTa as the language model have worse MRR than the models with BERT. Considering recall instead, it is more even with LogColRoBERTa3 performing a little better than LogColBERT, and LogColRoBERTa2 performing similarly at $K = 100$ but worse at $K = 10$ as compared to ColBERT.

Table 4.8: *Retrieval metrics for the retrieval models based on BERT and LogRoBERTa on the ELR evaluation dataset. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
ColBERT	**0.185**	**0.193**	**0.519**	0.765
LogColRoBERTa2	0.154	0.173	0.441	**0.778**
LogColBERT	**0.195**	**0.199**	**0.580**	0.765
LogColRoBERTa3	0.159	0.165	**0.580**	**0.778**

4.2.3 Fine-Tuning on Retrieval Datasets

The final experiments compare the impact of fine-tuning on the MS MARCO and ELR datasets. First, the impact of fine-tuning only on ELR is compared to only fine-tuning on MS MARCO. This impact is seen by comparing the performance of LogColRoBERTa1, which is only trained on ELR, and LogColRoBERTa2, which is only trained on MS MARCO.

The performance of these models on the MS MARCO evaluation dataset is seen in Table 4.9. From it, it is seen that the performance of only training on ELR is much lower than when only training on MS MARCO. LogColRoBERTa2 achieves a 914% higher MRR@10 and 694% higher Recall@10 relative to LogColRoBERTa1.

Table 4.9: *Recall for different retrieval models on the MS MARCO test set. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
LogColRoBERTa1	0.043	0.046	0.118	0.225
LogColRoBERTa2	**0.436**	**0.437**	**0.937**	**0.962**

There is a similar but smaller difference in performance when considering the ELR evaluation dataset instead. As seen in Table 4.10, fine-tuning only on the larger general domain MS MARCO dataset still leads to an improvement of MRR@10 by 28% and Recall@10 by 59%.

Some further improvements to performance are achieved when the retrieval model is first fine-tuned on MS MARCO, and then on ELR. The impact is seen in the final experiment when comparing the models only trained on MS MARCO: ColBERT and LogColRoBERTa2, to the models trained on both MS MARCO and ELR: LogColBERT and LogColRoBERTa3.

Table 4.10: *Recall for different retrieval models on the ELR evaluation dataset. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
LogColRoBERTa1	0.120	0.136	0.278	0.611
LogColRoBERTa2	**0.154**	**0.173**	**0.441**	**0.778**

Looking at the ELR evaluation dataset, it is seen in Table 4.11 that LogColBERT and LogColRoBERTa3 both achieve a higher MRR@10 compared to ColBERT and LogColRoBERTa2. For MRR@100 LogColBERT outperforms ColBERT while LogColRoBERTa2 outperforms LogColRoBERTa3. Recall at $K = 10$ is also higher, while for $K = 100$ it is identical. Overall, the two models fine-tuned further on ELR outperform the other two on the log retrieval task.

Table 4.11: *Retrieval metrics on the ELR dataset for the models that have been fine-tuned on MS MARCO only, or also on ELR. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
ColBERT	0.185	0.193	0.519	**0.765**
LogColBERT	**0.195**	**0.199**	**0.580**	**0.765**
LogColRoBERTa2	0.154	**0.173**	0.441	**0.778**
LogColRoBERTa3	**0.159**	0.165	**0.580**	**0.778**

The models that are further fine-tuned on ELR also don't show degraded or improved performance on the MS MARCO evaluation dataset. Comparing the models that are only fine-tuned on MS MARCO to those also fine-tuned on ELR in Table 4.12, shows that they have almost identical MRR and recall. The two models that have been further fine-tuned on ELR even show a small improvement over the other two.

Table 4.12: *Retrieval metrics on the MS MARCO dataset for the models that have been fine-tuned on MS MARCO only, or also on ELR. Higher is better.*

Model	MRR@10	MRR@100	Recall@10	Recall@100
ColBERT	**0.467**	0.467	**0.980**	0.997
LogColBERT	**0.467**	**0.468**	0.979	**0.998**
LogColRoBERTa2	0.436	0.437	0.937	0.962
LogColRoBERTa3	**0.437**	**0.438**	**0.938**	**0.965**

5 Discussion

This chapter discusses the results and the methodology used for obtaining the results.

5.1 Results

In this section, a discussion of the results in Chapter 4 is presented. The discussion is separated into two parts, the first discusses the results from the language model experiment which relates to the first research question, while the second part discusses the retriever experiments relating to the two other research questions.

5.1.1 Language Model

The purpose of continuing pretraining a language model on radio logs was to see if the model could learn better representations for the tokens in logs. The quantitative results in Section 4.1 clearly show that LogRoBERTa has learned to better predict masked log tokens, achieving both lower loss and higher accuracy at all evaluated K:s.

The quantitative results are confirmed by investigating the qualitative results. The LogRoBERTa model clearly outperforms the base RoBERTa model on common loglines that are well represented in the training data, such as the sample seen in Table 4.2. For this sample, LogRoBERTa predicts the correct token ”continues” with a very high 99.9% certainty, while RoBERTa is unable to. This result is to be expected given that the sample is well represented in the radio log training data but not in Wikipedia.

Where it gets more interesting is in the case where an unusual logline is evaluated, like the sample in Table 4.3. This sample logline is not present directly in

the training data, but despite this, the LogRoBERTa model still correctly predicts the masked token, though with a lower certainty than in the common case. By investigating the top-3 predictions, it is clear that LogRoBERTa is able to infer that the logline is a voltage reading. This can be compared to the predictions of the normal RoBERTa model, which is also able to understand that the logline is related to power, but fails to understand the connection to voltage. This suggests that the LogRoBERTa model has gained a deeper semantic understanding of the log context than RoBERTa, which it uses to infer the correct token.

The final question that needs to be evaluated for LogRoBERTa is whether the continued pretraining on radio logs has caused the model to forget its original domain knowledge. There is some evidence in the quantitative results in Section 4.1 that this has occurred, with LogRoBERTa having both higher loss and lower accuracy than RoBERTa on the Wikipedia dataset. It is also seen on the natural language sample in Table 4.4. For this sample, both RoBERTa and LogRoBERTa are able to predict the missing token is "ice" correctly. Furthermore, both models are also able to give reasonable top-3 predictions. It is seen however that LogRoBERTa has lost some certainty in its top prediction, and has shifted some of the probability over from "ice" to "whipped".

Clearly, the gain in performance on radio logs has come at some expense to the performance on natural language. The difference in the performance gain on the radio logs dataset is however greater than the decrease on Wikipedia, and given that the intended purpose of the model is to understand the semantic meaning of logs better, this trade-off is likely beneficial.

5.1.2 Retrieval Model

The retrievers are evaluated using several different experiments. First, the experiment comparing the representation-based paradigm to the late interaction paradigm is discussed, relating to the second research question. Then, the experiments on the ColBERT model relating to the third research question are discussed.

Retrieval Paradigm

In Section 4.2.1, a comparison of a representation-based model, SentenceBERT, is compared to the later-interaction model ColBERT. The results show that ColBERT outperforms SentenceBERT on the new log data domain, without being explicitly trained on it. This suggests that ColBERT is better able to generalise to new out-of-domain data. As described in Section-ColBERT, this is likely due to the fact that representation-based models like SentenceBERT reduce each document and query into two independent and "lossy" vector representations. ColBERT on the other hand preserves information with its late interaction mechanism by allowing interaction at the token level.

That ColBERT has more generalisable representations for queries and documents is especially relevant for specialised tasks like log retrieval, where the available training data is lacking. These generalisable representations can instead be

learned from larger general domain datasets such as MS MARCO and transferred to new domains more reliably than can be achieved with a representation-based approach.

When comparing SentenceBERT to ColBERT on the MS MARCO evaluation dataset, it is seen however that SentenceBERT outperforms ColBERT on the MRR metrics, achieving an MRR@10 of 0.860. This result is suspiciously high relative to ColBERT, and relative to what the original authors report. The authors report that ColBERT achieves an MRR@10 of 0.360 (Khattab and Zaharia, 2020), which for our evaluation dataset increases to 0.467. For SentenceBERT however, the authors report an MRR@10 of 0.323 (Reimers, 2021b), which increases to 0.860 for our dataset. An increase in performance might be expected due to subsampling the MS MARCO evaluation dataset, as described in Section 3.3, which for each query reduces the total number of irrelevant documents, however, such benefit should affect both models equally. Given that the original authors report MRR@10 in the 0.30 to 0.40 range, with ColBERT outperforming SentenceBERT, the result from this experiment is to be regarded suspiciously.

Different Language Models

The BERT language model that the original ColBERT uses has only been pretrained on natural text. It is interesting to see if replacing this language model with LogRoBERTa, which has been pretrained on logs as well, improves the model performance on the log retrieval task. As might be expected, the results in Section 4.2.2 show that replacing BERT with LogRoBERTa decreased performance on the MS MARCO evaluation dataset. As discussed in 5.1.1, the LogRoBERTa model has improved representations for log data, but slightly worse representations for natural text. It is therefore reasonable that the performance on MS MARCO decreases as a result.

An unexpected result was however that these improved representations do not result in an improved performance on the log retrieval task. The results show that substituting BERT for LogRoBERTa lowers the performance both when fine-tuning only on MS MARCO and when also fine-tuning on the ELR dataset. It is only when comparing recall for the higher values of K that the LogRoBERTa-based models show a small improvement over the BERT-based models. This suggests that the better token representations that LogRoBERTa provides for log data do not help the model on the retrieval task.

Fine-Tuning on Retrieval Datasets

The final experiments investigate the impact of fine-tuning the retrieval models on the two datasets, ELR and MS MARCO. From the experiments, it is clear that only fine-tuning on a small dataset like ELR is not sufficient. The model that was only trained on ELR showed abysmal performance on the MS MARCO, as can be expected, but also performed poorly on the ELR evaluation dataset. Interestingly, the performance on ELR is improved even if the model is instead only fine-tuned on MS MARCO. From this, it is clear that what the model learns from training on MS MARCO translates well to the log retrieval task.

As is to be expected, however, the models achieve the best performance when trained on both MS MARCO and ELR. The performance increase on ELR of further fine-tuning is clear, but not very large, with the difference being smaller than between only training on ELR and MS MARCO. It is worth noting however that further fine-tuning on ELR does not degrade performance on MS MARCO, as might be expected. This suggests that the tasks are either similar enough to not impact performance, or that the ELR dataset is too small for the model to start forgetting the original domain.

It is further interesting to note that the relative performance increase of further fine-tuning on ELR is less for the model with the LogRoBERTa language model. This discrepancy might be due to the language model impeding the learning of the retriever.

5.2 Method

One aspect of the method worth discussing is the ELR dataset. The dataset was gathered to help train the retrieval models for the log retrieval task, with the goal of helping the model bridge the gap from the normal retrieval task that might be learned from a general domain dataset like MS MARCO to the new specific log domain. However, the 249 training triplets and 27 evaluation relevancy pairs that were gathered are much smaller than the 39.8 million training triplets and 1000 evaluation relevancy pairs from MS MARCO. Had a larger dataset been collected, it likely could have yielded better results and conclusions.

Another aspect of the methodology that could be reevaluated is the choice of RoBERTa as the alternate language model, instead of the normal BERT model that ColBERT uses. While the authors of RoBERTa showed that the RoBERTa approach to pretraining outperforms the BERT approach on all evaluated tasks, it is possible that log retrieval is an exception. The poor performance of the LogColRoBERTa models compared to ColBERT may be due to RoBERTa being an unsuitable choice for this task.

6

Conclusions

In this chapter, the conclusions of the thesis are presented, along with recommendations for future work. The purpose of this thesis was to find a solution to help technicians and engineers troubleshoot radio logs. The investigation solution is a retrieval system capable of fast and intelligent search in logs.

A prerequisite to building a modern retrieval system is to have a suitable language model. The first research question therefore focuses on the possibility of teaching a language model better representations for log data. The experiments show that the new LogRoBERTa model has successfully learned to better represent log data than the original RoBERTa model. The new model even shows good performance for log data that is not commonly represented in the training data, indicating that it has gained an understanding of the data. This result also showed that the performance improvement on log data does degrade the performance slightly on the original natural text-domain.

The second research question investigates which of the retrieval paradigms is better suited for the log retrieval task. The two full-interaction paradigms were discarded out of hand due to their poor scaling performance, which left the representation-based similarity and late interaction paradigms to be evaluated. From the experiments, it is clear that while the representation-based similarity performs well on the original MS MARCO dataset, it does not generalise well to new data. Instead, the late interaction paradigm yields better performance on the log retrieval task. This finding is especially relevant for other retrieval tasks like this one, where the amount of training data available is small.

The final research question investigates how a retrieval model can be trained to search through logs. To investigate this, several models were trained using two different datasets and two different language models. The impact of replacing the original BERT model with the LogRoBERTa model that was further pretrained on logs was evaluated. The results of comparing these models show that replacing

BERT with LogRoBERTa did not improve performance, and even degraded it. Despite LogRoBERTa providing the retrieval model with better and more contextualised token representations, these were not helpful in learning the log retrieval task.

While replacing the language model did not improve performance, fine-tuning on the retrieval datasets did. Specifically, fine-tuning on a large general domain dataset like MS MARCO transferred well to the log retrieval task. The size of the dataset is important though, as the model trained only the small ELR dataset had poor performance. The best performance on the log retrieval task is achieved by first fine-tuning the retrieval models on the larger MS MARCO dataset, followed by the small ELR dataset.

6.1 Future Work

The retrieval method evaluated in this thesis shows great promise, but performance is likely limited due to a lack of training data. Future work should therefore focus on collecting more data for the new domain, and perhaps also consider using synthetic data methods and distillation learning. Another interesting direction that can be investigated is considering how a retriever can handle queries that require fetching a batch of results based on some interdependencies between them, such as power readings that are increasing over time. Question-answering systems commonly tackle this type of problem with a multi-hop reasoning approach, which might be tried in this domain.

Bibliography

Chase, Harrison (Oct. 2022). *LangChain*. URL: https://github.com/langchain-ai/langchain (visited on 05/22/2024).

Cover, Thomas M. and Joy A. Thomas (Apr. 2005). "Elements of Information Theory". In: Wiley, p. 19. ISBN: 9780471748823. DOI: 10.1002/047174882x. URL: http://dx.doi.org/10.1002/047174882X.

Craswell, Nick (2009). "Mean Reciprocal Rank". In: *Encyclopedia of Database Systems*. Ed. by LING LIU and M. TAMER ÖZSU. Boston, MA: Springer US, p. 1703. ISBN: 978-0-387-39940-9. DOI: 10.1007/978-0-387-39940-9_488. URL: https://doi.org/10.1007/978-0-387-39940-9_488.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *North American Chapter of the Association for Computational Linguistics*. URL: https://api.semanticscholar.org/CorpusID:52967399.

Douze, Matthijs, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou (2024). "The Faiss library". In: arXiv: 2401.08281 [cs.LG].

Facebook AI (2019). *RoBERTa large model*. URL: https://huggingface.co/FacebookAI/roberta-large (visited on 04/30/2024).

Guo, Jiafeng, Yixing Fan, Qingyao Ai, and W. Bruce Croft (2016). "A Deep Relevance Matching Model for Ad-hoc Retrieval". In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. CIKM '16. New York, NY, USA: Association for Computing Machinery, 55–64. ISBN: 9781450340731. DOI: 10.1145/2983323.2983769. URL: https://doi.org/10.1145/2983323.2983769.

Gururangan, Suchin, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith (July 2020). "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault. Online: Association for Computational Linguistics, pp. 8342–8360. DOI: 10.18653/v1/2020.acl-main.740. URL: https://aclanthology.org/2020.acl-main.740.

Hambarde, Kailash A. and Hugo Proença (2023). "Information Retrieval: Recent Advances and Beyond". In: *IEEE Access* 11, pp. 76581–76604. DOI: 10.1109/ACCESS.2023.3295776.

Jiang, Albert Q. et al. (2023). *Mistral 7B*. arXiv: 2310.06825 [cs.CL].

Khattab, Omar and Matei Zaharia (2020). "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT". In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '20. New York, NY, USA: Association for Computing Machinery, 39–48. ISBN: 9781450380164. DOI: 10.1145/3397271.3401075. URL: https://doi.org/10.1145/3397271.3401075.

Liu, Yang and Mirella Lapata (Nov. 2019). "Text Summarization with Pretrained Encoders". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan. Hong Kong, China: Association for Computational Linguistics, pp. 3730–3740. DOI: 10.18653/v1/D19-1387. URL: https://aclanthology.org/D19-1387.

Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov (2019). "RoBERTa: A Robustly Optimized BERT Pretraining Approach". In: *ArXiv* abs/1907.11692. URL: https://api.semanticscholar.org/CorpusID:198953378.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv: 1301.3781 [cs.CL].

Nayak, Pandu (Oct. 2019). *Understanding searches better than ever before*. URL: https://blog.google/products/search/search-language-understanding-bert/ (visited on 05/22/2024).

Nguyen, Tri, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng (Nov. 2016). *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. URL: https://www.microsoft.com/en-us/research/publication/ms-marco-human-generated-machine-reading-comprehension-dataset/.

OpenAI et al. (2024). *GPT-4 Technical Report*. arXiv: 2303.08774 [cs.CL].

Paszke, Adam et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. arXiv: 1912.01703 [cs.LG].

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning (2014). "GloVe: Global Vectors for Word Representation". In: *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543. URL: http://www.aclweb.org/anthology/D14-1162.

Powers, David (Jan. 2008). "Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness & Correlation". In: *Mach. Learn. Technol.* 2.

Reimers, Nils (2021a). *MSMARCO MiniLM L6*. URL: https://huggingface.co/sentence-transformers/msmarco-MiniLM-L6-cos-v5 (visited on 05/12/2024).

— (2021b). *MSMARCO Models*. URL: https://www.sbert.net/docs/pretrained-models/msmarco-v5.html (visited on 05/12/2024).

— (2021c). *Sentence Transformers*. URL: https://www.sbert.net/ (visited on 05/12/2024).

Reimers, Nils and Iryna Gurevych (Nov. 2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan. Hong Kong, China: Association for Computational Linguistics, pp. 3982–3992. DOI: 10.18653/v1/D19-1410. URL: https://aclanthology.org/D19-1410.

Robertson, Stephen, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford (Jan. 1995). "Okapi at TREC-3". In: *Overview of the Third Text REtrieval Conference (TREC-3)*. Gaithersburg, MD: NIST, pp. 109–126. URL: https://www.microsoft.com/en-us/research/publication/okapi-at-trec-3/.

Taylor, Wilson L (1953). ""Cloze procedure": A new tool for measuring readability". In: *Journalism quarterly* 30.4, pp. 415–433.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL].

Wikimedia Foundation (2018). *Wikimedia Downloads*. URL: https://dumps.wikimedia.org (visited on 05/12/2024).

Wolf, Thomas et al. (2020). *HuggingFace's Transformers: State-of-the-art Natural Language Processing*. arXiv: 1910.03771 [cs.CL].

Appendix

A

Results

A.1 Language Model

Table A.1: *Accuracy@N for the two language models on both datasets, for different N.*

Dataset	Model	N=1	N=3	N=5	N=10	N=25	N=50
Radio logs	RoBERTa	0.408	0.473	0.498	0.542	0.630	0.691
	LogRoBERTa	**0.854**	**0.893**	**0.911**	**0.942**	**0.967**	**0.985**
Wikipedia	RoBERTa	**0.649**	**0.775**	**0.802**	**0.837**	**0.875**	**0.896**
	LogRoBERTa	0.464	0.610	0.663	0.722	0.779	0.814

Table A.2: *Cross-Entropy Loss for the two language models on the evaluation datasets.*

Dataset	Model	Loss
Wikipedia	RoBERTa	**1.65593**
	LogRoBERTa	2.67029
Radio logs	RoBERTa	3.60516
	LogRoBERTa	**0.44931**

A.2 Retrieval Metrics

The full results for the different retrieval models are presented. First, the performance on MSMARCO is shown followed by the performance on the Radio logs dataset.

A.2.1 MSMARCO

The retrieval models performance on MS-MARCO are presented in Tables A.3 and A.4.

Table A.3: *Mean Reciprocal Rank (MRR) for the different retrieval models on the MSMARCO evaluation dataset.*

Model	MRR@10	MRR@25	MRR@100	MRR@500	MRR@1K
SentenceBERT	**0.860**	**0.861**	**0.861**	**0.861**	**0.861**
ColBERT	0.467	0.467	0.467	0.467	0.467
LogColBERT1	0.467	0.468	0.468	0.468	0.468
LogColBERT2	0.043	0.045	0.046	0.047	0.047
LogColBERT3	0.436	0.437	0.437	0.437	0.437
LogColBERT4	0.437	0.438	0.438	0.438	0.438

Table A.4: *Recall for the different retrieval models on the MSMARCO evaluation dataset.*

Model	Recall@10	Recall@25	Recall@100	Recall@500	Recall@1K
SentenceBERT	0.967	0.984	0.996	0.997	**0.999**
ColBERT	**0.980**	**0.991**	0.997	**0.998**	0.998
LogColBERT1	0.979	0.990	**0.998**	**0.998**	0.998
LogColBERT2	0.118	0.155	0.225	0.305	0.322
LogColBERT3	0.937	0.954	0.962	0.967	0.967
LogColBERT4	0.938	0.954	0.965	0.970	0.970

A.2.2 Radio Logs

The retrieval models performance on the radio logs dataset is presented in Tables A.5 and A.6.

Table A.5: *Mean Reciprocal Rank (MRR) for the different retrieval models on the ELR evaluation dataset.*

Model	MRR@10	MRR@25	MRR@100	MRR@500	MRR@1K
SentenceBERT	0.148	0.148	0.154	0.155	0.156
ColBERT	0.185	0.185	0.193	0.193	0.193
LogColBERT1	**0.195**	**0.195**	**0.199**	**0.199**	**0.199**
LogColBERT2	0.120	0.134	0.136	0.137	0.137
LogColBERT3	0.154	0.173	0.173	0.174	0.174
LogColBERT4	0.159	0.165	0.165	0.165	0.165

Table A.6: *Recall for the different retrieval models on the ELR evaluation dataset.*

Model	Recall@10	Recall@25	Recall@100	Recall@500	Recall@1K
SentenceBERT	0.250	0.250	0.365	0.821	**0.988**
ColBERT	0.519	0.543	0.765	**0.825**	0.825
LogColBERT1	**0.580**	0.654	0.765	0.813	0.813
LogColBERT2	0.278	0.543	0.611	0.778	0.778
LogColBERT3	0.441	**0.765**	**0.778**	**0.825**	0.825
LogColBERT4	**0.580**	**0.765**	**0.778**	0.778	0.778

B

BERT

B.1 Computational cost

The following is a derivation and computation of the number of operations required to perform inference for one input to BERT. BERT is a transformer model, so first a general derivation is performed for a single transformer block outlined in Vaswani et al. (2017), followed by the computation for BERT.

The input to a transformer is a matrix $X \in \mathbb{R}^{n \times d}$, where n is the input length (number of tokens) and d is the hidden dimension size. First, self-attention is computed. A transformer block consists of multiple self-attention heads and for each, three matrices

$$Q = XW_q, \quad K = XW_k, \text{ and } \quad V = XW_v \tag{B.1}$$

need to be computed. Multiplying the $W_q \in \mathbb{R}^{d \times d_k}$, $W_k \in \mathbb{R}^{d \times d_k}$ and $W_v \in \mathbb{R}^{d \times d_v}$ matrices with X results in $O(n \cdot d \cdot (d_k + d_k + d_v))$ operations. Then, the attention is computed through

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d}})V, \tag{B.2}$$

which contains two matrix multiplication and a row-wise softmax yielding another $O(n^2 \cdot (d_k + d_v + 1))$. This is performed for each of the h number of attention heads. The multi-head is then computed by

$$\text{MultiHead} = \text{Concat}(\text{Attention}_1, ..., \text{Attention}_h)W_0, \tag{B.3}$$

with $W^0 \in \mathbb{R}^{hd_v \times d}$, resulting in $O(n^2 \cdot h \cdot d_v)$. The result, renamed to $x \in \mathbb{R}^{n \times d}$, is then passed through a feed-forward network. The network consists of

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \tag{B.4}$$

where $W_1 \in \mathbb{R}^{d \times dff}$ and $b_1 \in \mathbb{R}^{n \times dff}$ are the first layer operations, and $W_2 \in \mathbb{R}^{dff \times d}$ and $b_2 \in \mathbb{R}^{n \times d}$ the second layer operations. Adding up the operations is then $O(2 \cdot n \cdot d \cdot dff + n \cdot dff + n \cdot d)$. The total estimated time complexity of a single transformer block is then $O(n * d * (2 * d_k + d_v + 1 + 2 * dff) + n^2 * h * (d_k + 2 * d_v + 1) + n * dff)$. BERT uses 12 blocks, each with $d = 768$, $d_k = d_v = 64$, $n = 512$, $h = 12$ and dff = 3072, yielding an estimated total number of 37.2 billion operations.